Informatics

Programming and using R
Scalars

Claudio Sartori
Department of Computer Science and Engineering
claudio.sartori@unibo.it
https://www.unibo.it/sitoweb/claudio.sartori/

slides © Pearsons – Fluency with Information Technology – Snyder – Ch.09

Introduction

- RStudio allows the user to run R in a more user-friendly environment. It is open-source (i.e. free) and available at http://www.rstudio.com/
- For R related tutorials and/or resources see the following links:
- http://dss.princeton.edu/training/
- http://libguides.princeton.edu/dss

RStudio screen

RStudio

File Edit Code View Plots Session Project Build Tools Help

Go to file/function

Project: (None)

Console H:/MyData/RFiles/

```
R version 3.0.0 (2013-04-03) -- "Masked Marvel"
Copyright (C) 2013 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64 (64-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> getwd()
[1] "H:/MyData/RFiles"
> 5*5
[1] 25
> A <- matrix(c(1,2,3,4,5,6,7,8), nrow=4, ncol=2)
> A
     [,1] [,2]
[1,]    1    5
[2,]    2    6
[3,]    3    7
[4,]    4    8
> B <- matrix(c(1,2,3,4,5,6,7,8), nrow=4, ncol=2, byrow=TRUE)
> B
     [,1] [,2]
[1,]    1    2
[2,]    3    4
[3,]    5    6
[4,]    7    8
>
```

The **console** is where you can type commands and see output

Workspace History

Import Dataset

Data	
A	4x2 double matrix
B	4x2 double matrix

The **workspace** tab shows all the active objects (see next slide). The history tab shows a list of commands used so far.

Files Plots Packages Help

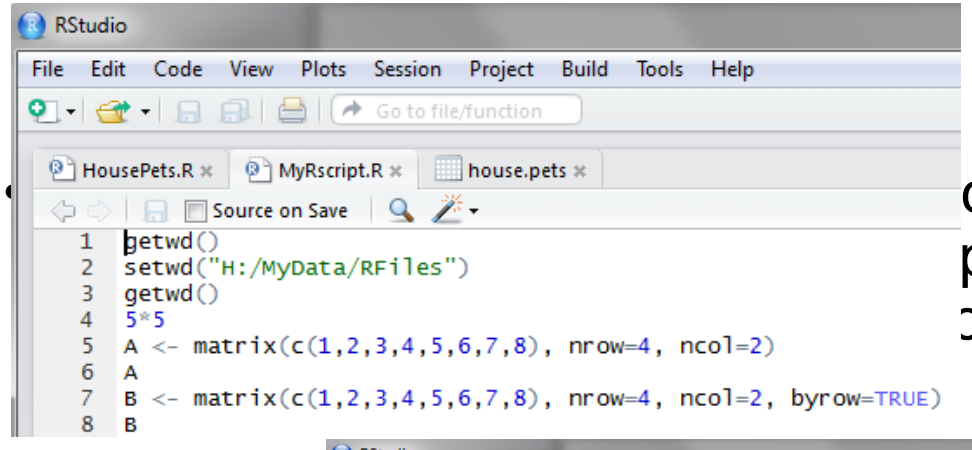
New Folder Delete Rename More

H: > MyData > RFiles

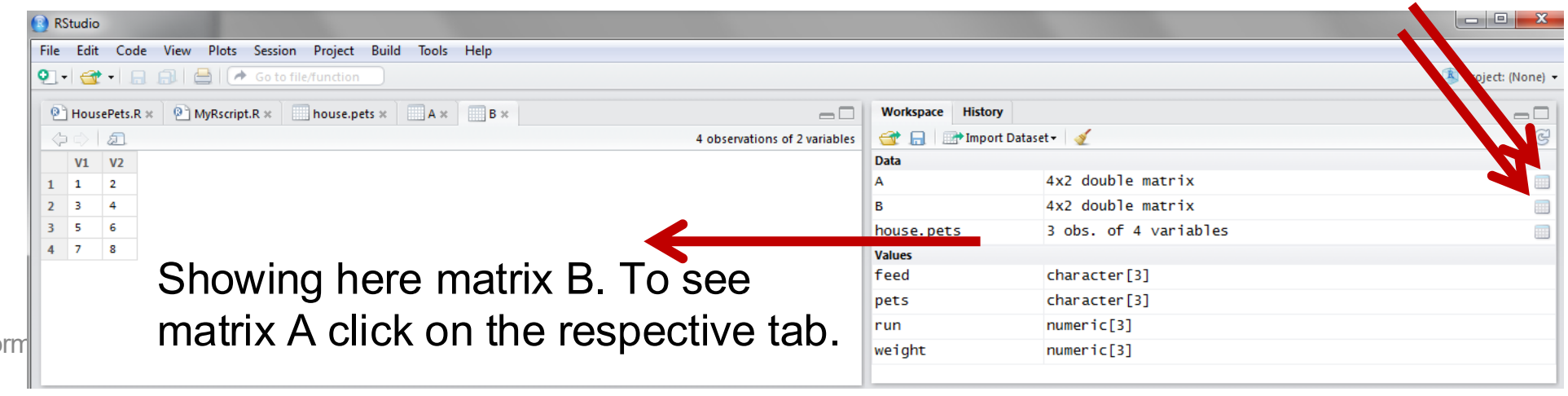
Name	Size	Modified
..		
.Rhistory	34 bytes	Aug 23, 2013, 1:26 PM

The **files** tab shows all the files and folders in your default workspace as if you were on a PC/Mac window. The **plots** tab will show all your graphs. The **packages** tab will list a series of packages or add-ons needed to run certain processes. For additional info see the **help** tab

Workspace tab



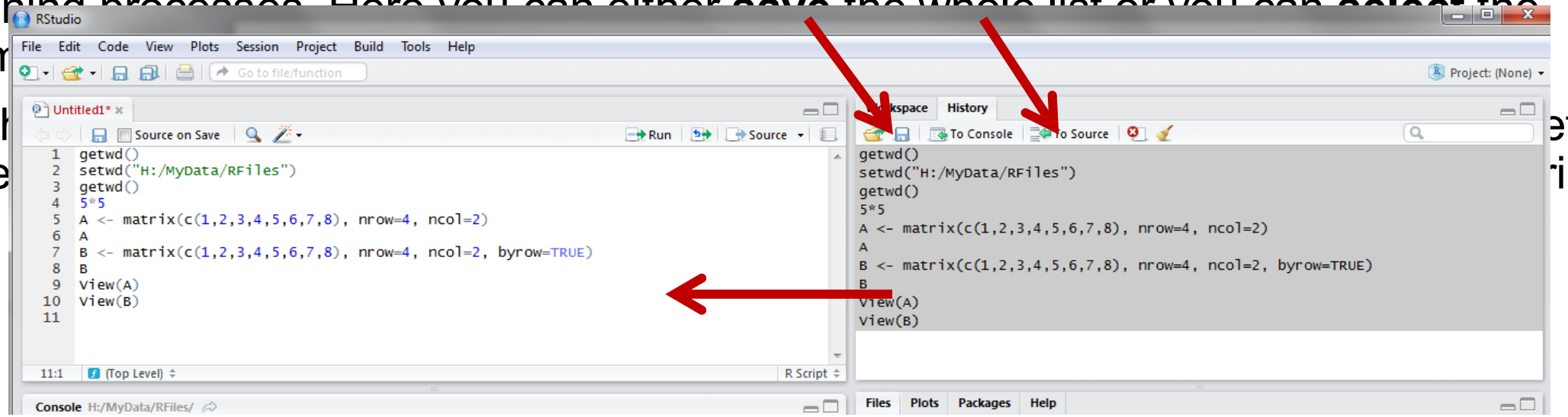
ct, value, function or anything you create
ple below, if you click on the dotted squares
o the left



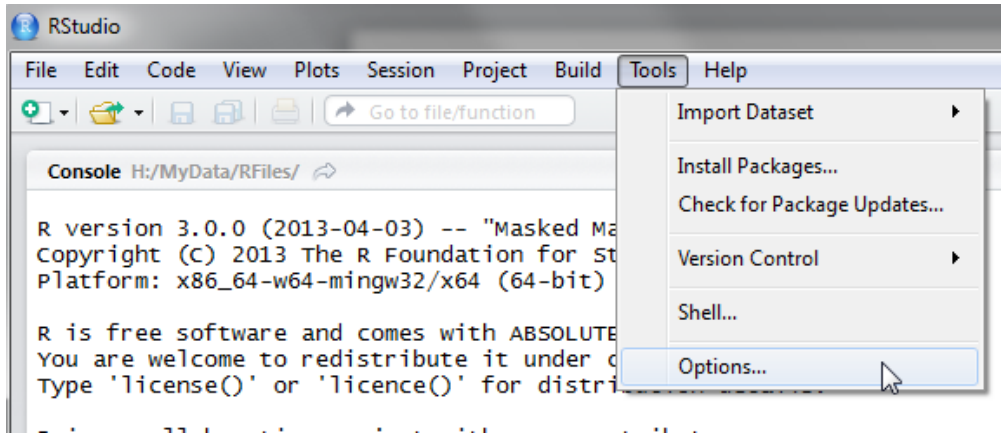
Showing here matrix B. To see matrix A click on the respective tab.

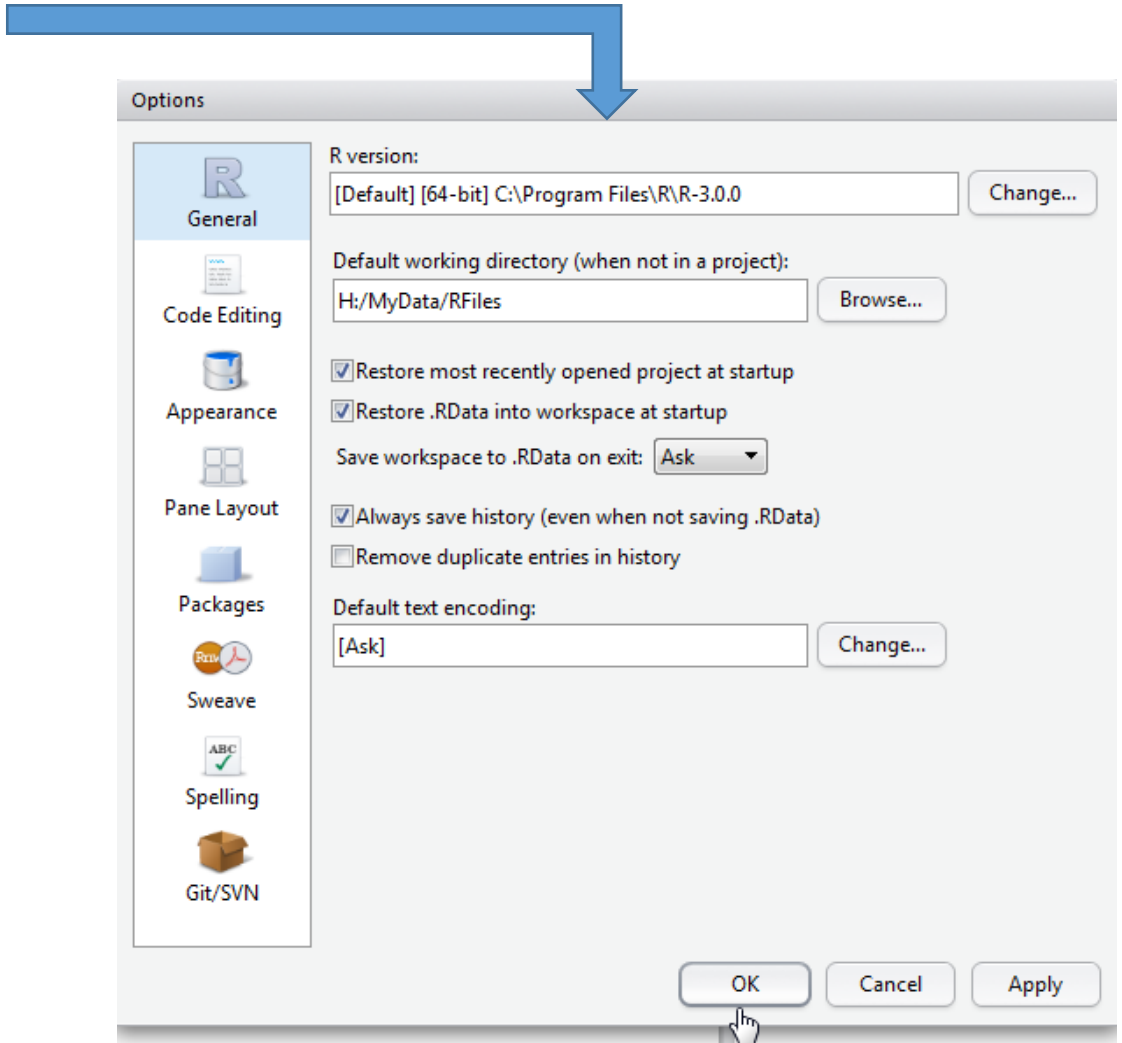
History tab

- The history tab keeps a record of all previous commands. It helps when testing and runn
com
- In th eft will
ope ript.



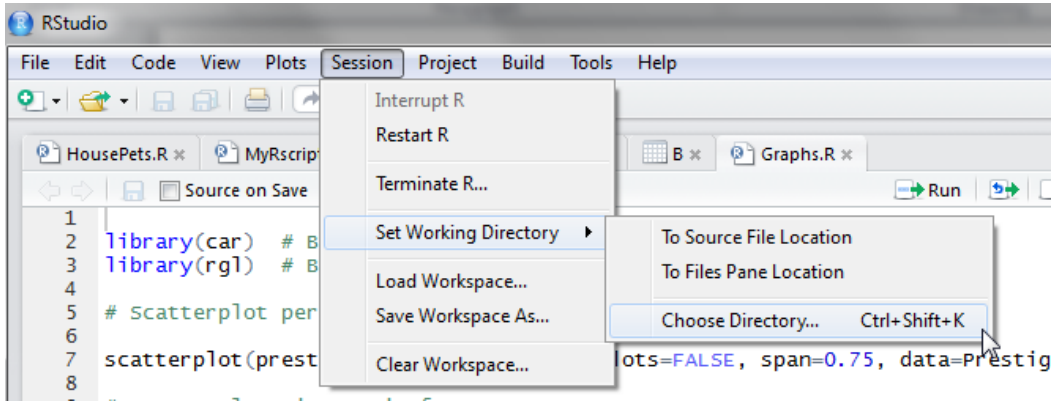
Setting a default working directory

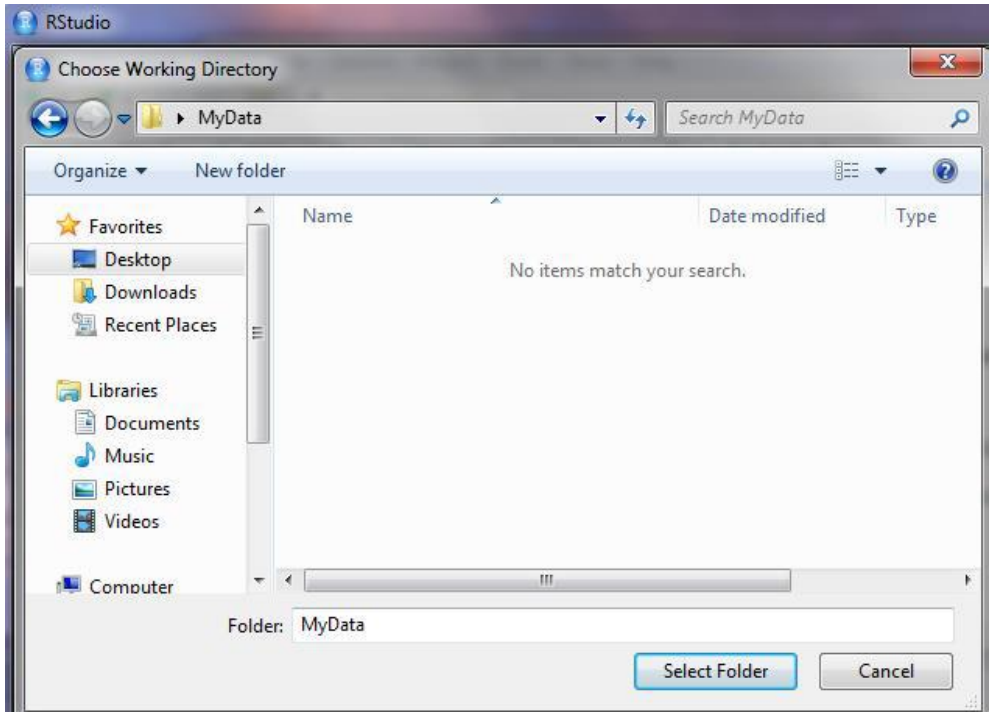




- Every time you open RStudio, it goes to a default directory. You can change the default to a folder where you have your datafiles so you do not have to do it every time. In the menu go to `Tools -> Options`

Changing the working directory



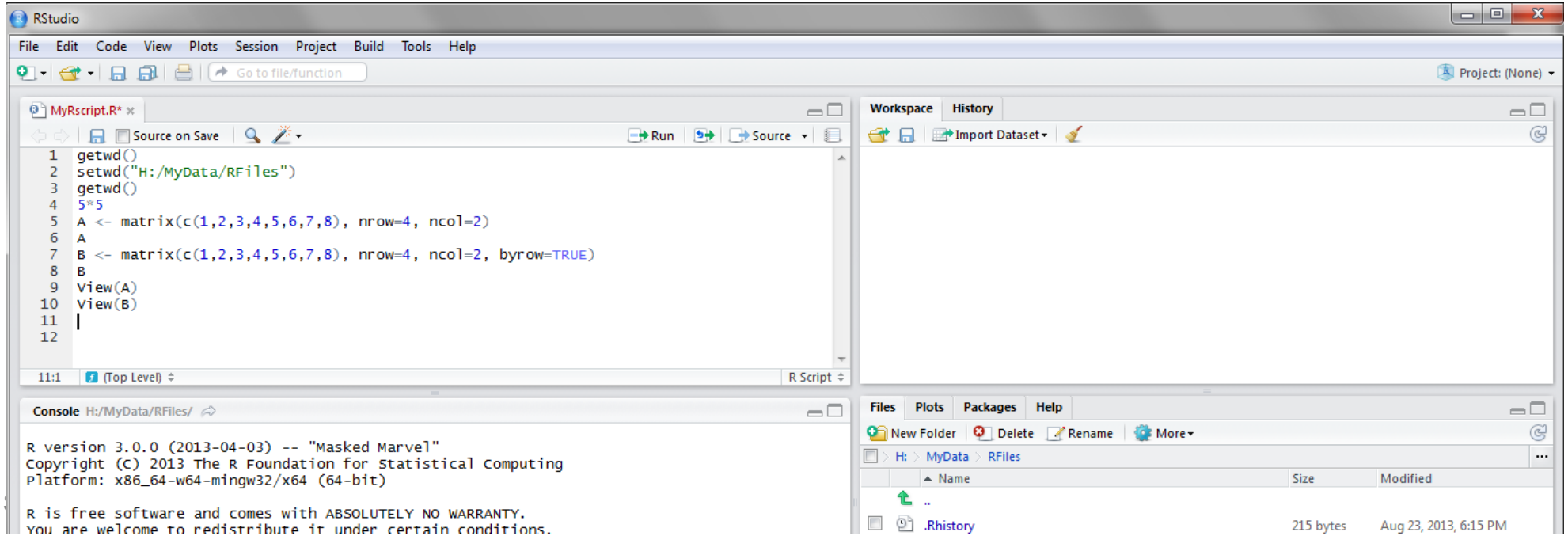


- If you have different projects you can change the working directory for that session, see above. Or you can type:
- `# Shows the working directory (wd)`
- `getwd()`
- `# Changes the wd`
- `setwd("C:/myfolder/data")`

R Script (I)

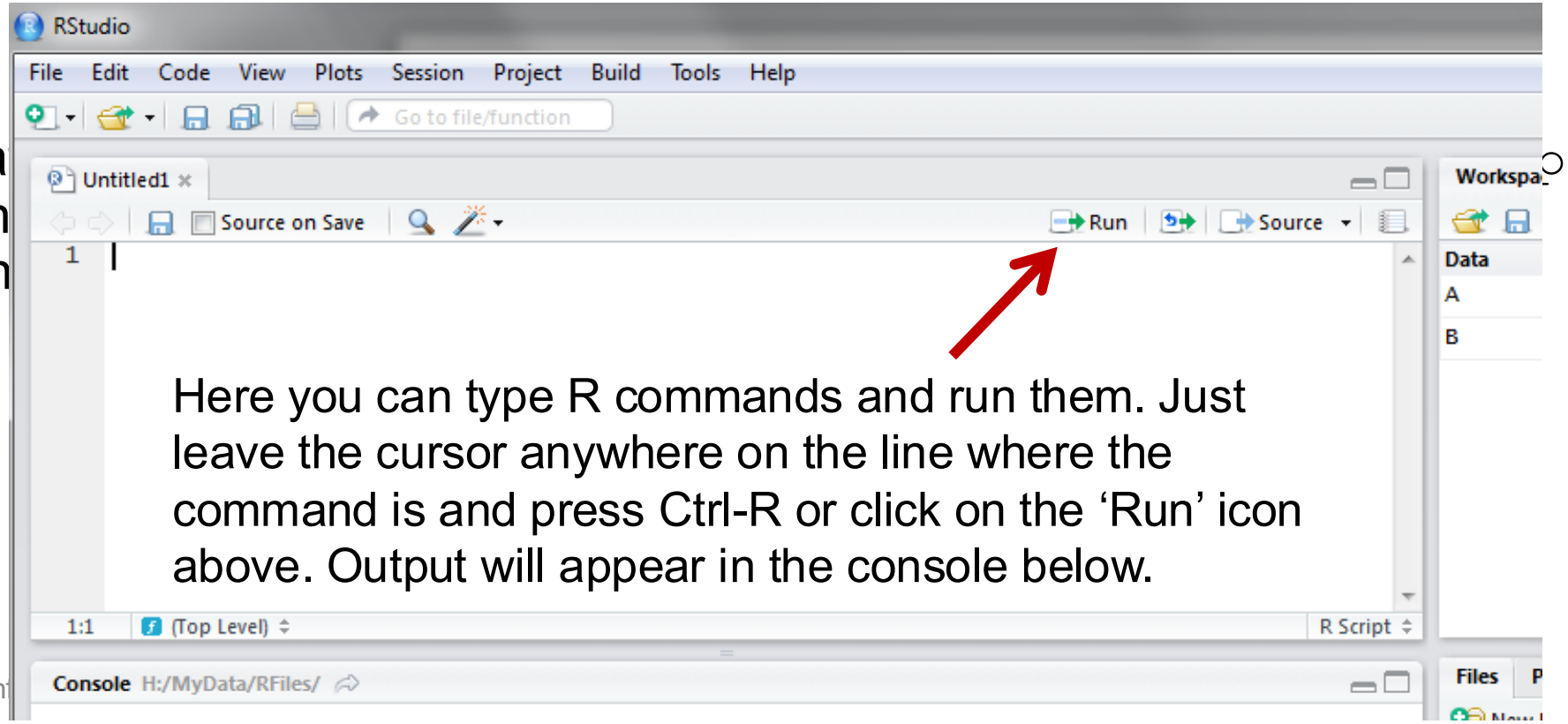
- The usual Rstudio screen has four windows:

1. Console.
2. Workspace and history.
3. Files, plots, packages and help.
4. The R script(s) and data view. The R script is where you keep a record of your work.



R Script (II)

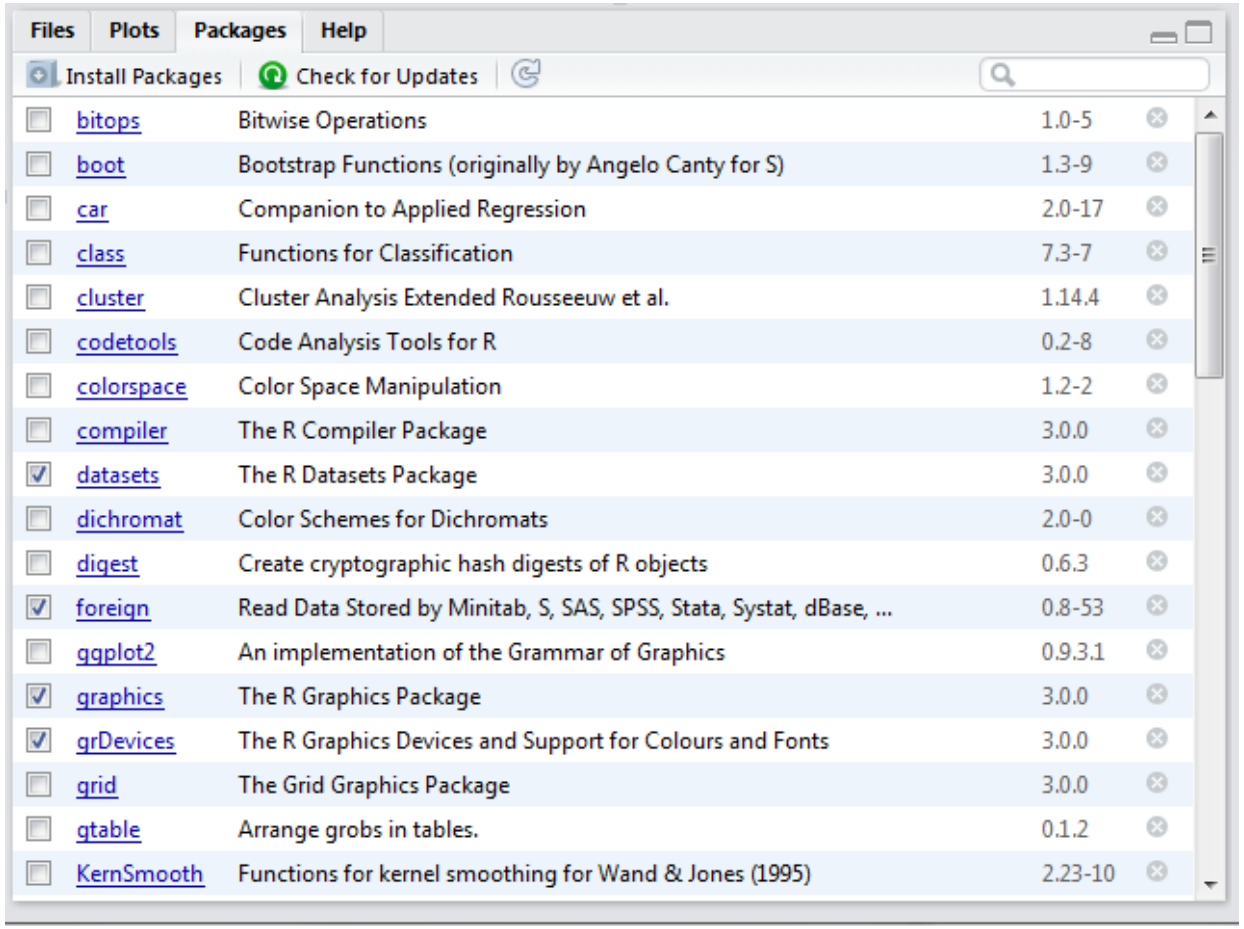
- To crea ot, or click on Ctrl+Sh



Here you can type R commands and run them. Just leave the cursor anywhere on the line where the command is and press Ctrl-R or click on the 'Run' icon above. Output will appear in the console below.

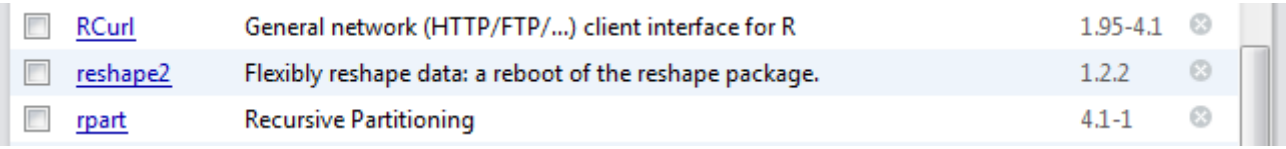
Packages tab

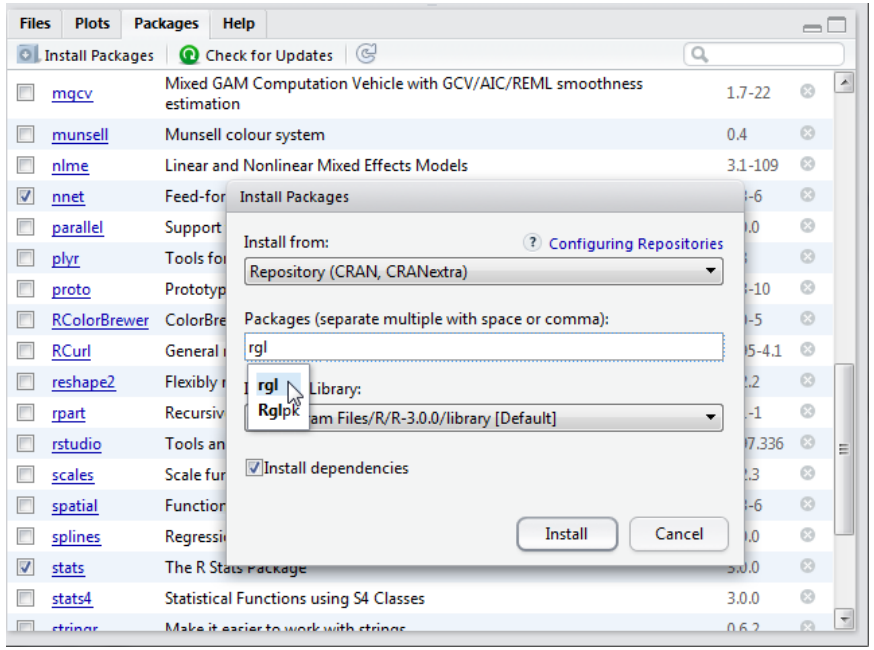
- The **package tab** shows the list of add-ons included in the installation of RStudio. If checked, the package is loaded into R, if not, any command related to that package won't work, you will need select it. You can also install other add-ons by clicking on the 'Install Packages' icon.
- Another way to activate a package is by typing, for example, `library(foreign)`. This will automatically check the `--foreign` package (it helps bring data from proprietary formats like Stata, SAS or SPSS).



Installing a package

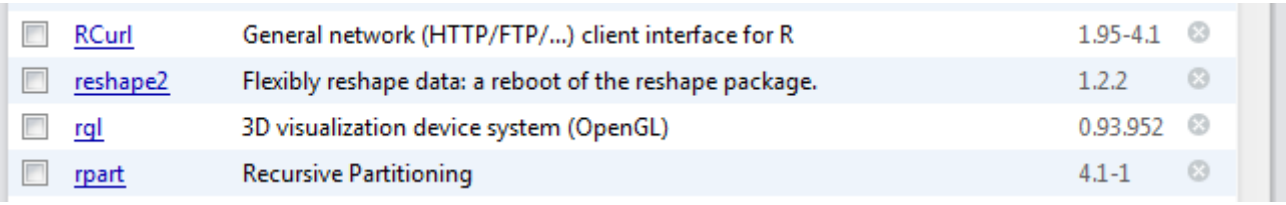
Before





- We are going to install the package `-rgl` (useful to plot 3D images). It does not come with the original R install.
- Click on "`Install Packages`", write the name in the pop-up window and click on "`Install`".

After



R Basics

How it works

100xp

In the editor on the right you should type R code to solve the exercises. When you hit the 'Submit Answer' button, every line of code is interpreted and executed by R and you get a message whether or not your code was correct. The output of your R code is shown in the console in the lower right corner.

R makes use of the `#` sign to add comments, so that you and others can understand what the R code is about. Just like Twitter! Comments are not run as R code, so they will not influence your result. For example, *Calculate 3 + 4* in the editor on the right is a comment.

You can also execute R commands straight in the console. This is a good way to experiment with R code, as your submission is not checked for correctness.

Instructions

- In the editor on the right there is already some sample code. Can you see which lines are actual R code and which are comments?
- Add a line of code that calculates the sum of 6 and 12, and hit the 'Submit Answer' button.

script.R

```
# Calculate 3 + 4


# Calculate 6 + 12

```

Little arithmetics with R

100xp

In its most basic form, R can be used as a simple calculator. Consider the following arithmetic operators:

- Addition: `+`
- Subtraction: `-`
- Multiplication: `*`
- Division: `/`
- Exponentiation: `^`
- Modulo: `%%`

The last two might need some explaining:

- The `^` operator raises the number to its left to the power of the number to its right: for example `3^2` is 9.
- The modulo returns the remainder of the division of the number to the left by the number on its right, for example 5 modulo 3 or `5 %% 3` is 2.

With this knowledge, follow the instructions below to complete the exercise.

Instructions

- Type `2^5` in the editor to calculate 2 to the power 5.
- Type `28 %% 6` to calculate 28 modulo 6.
- Click 'Submit Answer' and have a look at the R output in the console.
- Note how the `#` symbol is used to add comments on the R code.

script.R

```
# An addition


# A subtraction


# A multiplication


 # A division


# Exponentiation


# Modulo

```

Variable assignment

100xp

A basic concept in (statistical) programming is called a **variable**.

A variable allows you to store a value (e.g. 4) or an object (e.g. a function description) in R. You can then later use this variable's name to easily access the value or the object that is stored within this variable.

You can assign a value 4 to a variable `my_variable` with the command

```
my_variable <- 4
```

Instructions

Over to you: complete the code in the editor such that it assigns the value 42 to the variable `x` in the editor. Click 'Submit Answer'. Notice that when you ask R to print `x`, the value 42 appears.

script.R

```
# Assign the value 42 to 'x'


# Print out the value of the variable 'x'

```

Variable assignment (2)

100xp

Suppose you have a fruit basket with five apples. As a data analyst in training, you want to store the number of apples in a variable with the name `my_apples` .

Instructions

- Type the following code in the editor: `my_apples <- 5` . This will assign the value 5 to `my_apples` .
- Type: `my_apples` below the second comment. This will print out the value of `my_apples` .
- Click 'Submit Answer', and look at the console: you see that the number 5 is printed. So R now links the variable `my_apples` to the value 5.

script.R

```
# Assign the value 5 to the variable called 'my_apples'


# Print out the value of the variable 'my_apples'

```

Variable assignment (3)

100xp

Every tasty fruit basket needs oranges, so you decide to add six oranges. As a data analyst, your reflex is to immediately create the variable `my_oranges` and assign the value 6 to it. Next, you want to calculate how many pieces of fruit you have in total. Since you have given meaningful names to these values, you can now code this in a clear way:

```
my_apples + my_oranges
```

Instructions

- Assign to `my_oranges` the value 6.
- Add the variables `my_apples` and `my_oranges` and have R simply print the result.
- Combine the variables `my_apples` and `my_oranges` into a new variable `my_fruit`, which is the total amount of fruits in your fruit basket.

script.R

```
# Assign a value to the variables called 'my_apples' and 'my_oranges'


# Add these two variables together and print the result


# Create the variable 'my_fruit'

```

Apples and oranges

100xp

Common knowledge tells you not to add apples and oranges. But hey, that is what you just did, no :-)? The `my_apples` and `my_oranges` variables both contained a number in the previous exercise. The `+` operator works with numeric variables in R. If you really tried to add "apples" and "oranges", and assigned a text value to the variable `my_oranges` (see the editor), you would be trying to assign the addition of a numeric and a character variable to the variable `my_fruit`. This is not possible.

Instructions

- Click 'Submit Answer' and read the error message. Make sure to understand why this did not work.
- Adjust the code so that R knows you have 6 oranges and thus a fruit basket with 11 pieces of fruit.

script.R

```
# Assign a value to the variable called 'my_apples'


# Print out the value of 'my_apples'


# Assign a value to the variable 'my_oranges' and print it out


# New variable that contains the total amount of fruit


```

Basic data types in R

100xp

R works with numerous data types. Some of the most basic types to get started are:

- Decimals values like `4.5` are called **numerics**.
- Natural numbers like `4` are called **integers**. Integers are also numerics.
- Boolean values (`TRUE` or `FALSE`) are called **logical** (`TRUE` can be abbreviated to `T` and `FALSE` to `F`).
- Text (or string) values are called **characters**.

Note how the quotation marks on the right indicate that "some text" is a character.

Instructions

Change the value of the:

- `my_numeric` variable to `42` .
- `my_character` variable to `"forty-two"` . Note that the quotation marks indicate that `"forty-two"` is a character.
- `my_logical` variable to `FALSE` .

Note that R is case sensitive!

script.R

```
# What is the answer to the universe?


# The quotation marks indicate that the variable is of type character
```

What's that data type?

100xp

Do you remember that when you added `5 + "six"`, you got an error due to a mismatch in data types? You can avoid such embarrassing situations by checking the data type of a variable beforehand. You can do this as follows:

```
class(some_variable_name)
```

Instructions

Complete the code in the editor and print the classes of `my_numeric`, `my_character` and `my_logical` to the console.

script.R

```
# Declare variables of different types



# Check which type these variables have:
```

Examples of computations

1. Arithmetic & Precedence of operators

- Set a = 7, b = 3, c = 2
  - Compute: a + b * c^2
  - Compute: (a + b) * c^2

2. Quadratic Formula

- Set A = 1, B = -5, C = 6
  - Compute the smaller root using the quadratic formula

3. Distance Between Two Points

- Set x1 = 2, y1 = 5
  - Set x2 = 7, y2 = 1
  - Compute Euclidean distance

4. Geometry

- Hypotenuse: $p = 9$, $q = 12$
  - Circle: $r = 4.5$
  - Compute hypotenuse, circumference, and area

5. Compound Interest

- P = 1000, r = 0.04, t = 3 years
  - Compounded monthly
  - Compute final amount

6. Logarithms & Trigonometry

- Set x = 10
  - Compute log(x), log10(x)
  - Verify log10(x) = log(x)/log(10)
  - Set theta = pi/6
  - Compute sin(theta)^2 + cos(theta)^2

7. Exponential Growth

- N0 = 500, k = 0.12, t = 8
  - Compute N = N0 * exp(k*t)

8. Rounding & Scientific Notation

- Set x = 123.456789
  - Compute round, floor, ceiling, trunc
  - Compute 3.2e5 / 7.5e-2
  - Format result in scientific notation

9. Standardization & BMI

- Given x = 73, mu = 65, sigma = 8
  - Compute z-score

- BMI: kg = 72, m = 1.78
  - Compute BMI

10. Time & Currency Conversion

- Convert 3671 seconds into hours, minutes, seconds

- EUR = 250, rate = 1.09, fee = 0.015
  - Compute USD after fee

Advanced Base R Programming Exercises

Scalar-Only

The purpose is not to explain the meaning of the formulas, but to implement the given formulas in R

1. Logistic Transformation

- Given $x = 1.75$
  - Compute logistic value: $1 / (1 + \exp(-x))$
  - Interpret as probability

2. Continuous Compounding vs Discrete

- P = 1500, r = 0.06, t = 5
  - Compute discrete annual: P * (1 + r)^t
  - Compute continuous: P * exp(r*t)
  - Compare difference

3. Numerical Stability

- Set x = 1e-8
  - Compute log(1 + x)
  - Compute log1p(x)
    - check the help to see the definition of log1p
  - Compute the difference

4. Standard Normal Density

- Given x = 0.8
  - Compute density manually:
  - (1/sqrt(2*pi)) * exp(-x^2/2)

5. Z-score Back-Transformation

- Given z = 1.2, mu = 50, sigma = 10
  - Recover original value: mu + z*sigma

6. Root Mean Square (Two Values)

- Given a = 3, b = 7
  - Compute RMS: sqrt((a^2 + b^2)/2)

7. Log-Likelihood (Single Observation)

- Assume normal distribution
  - x = 5, mu = 4.2, sigma = 1.5
  - Compute log-likelihood manually

8. Elasticity Calculation

- Q1 = 120, Q2 = 150
  - P1 = 10, P2 = 12
  - Compute the midpoint elasticity formula
  - (Q2-Q1) / (Q1+Q2)/2 / (P2-P1) / (P1+P2)/2

9. Half-Life Calculation

- Given decay rate $k = 0.35$
  - Compute half-life: $\log(2)/k$

10. Newton Step (Single Iteration)

- For $f(x) = x^2 - 5$
  - Start $x_0 = 2$
  - Compute one Newton update:
  - $x_1 = x_0 - f(x_0)/(2*x_0)$

11. Entropy (Binary Case)

- Given p = 0.3
  - Compute entropy:
  - -p*log(p) - (1-p)*log(1-p)